This application is called Lessly. It is a Java web application that helps users track their belongings and manage impulsive shopping habits. Users can log items they own, track how often they use them, and receive suggestions on whether to keep, donate, sell, or discard items based on usage and value. A report summarizes spending habits and decluttering progress, with a real AI-generated motivational summary powered by Google Gemini.

The features are:

- Adding clothing, electronics, and perishable items in your inventory to track.
- Items saved in a MySQL database via MAMP.
- Has a status for each item, which is determined based on when items are added or used.
- Flagging items if they are unused for 30+ days or expired.
- Users can also manually alter the status and prevent auto-updating the item.
- Report that shows flagged items, removal suggestions, and spending summary.
- Expired items are always flagged.
- Suggested removals show priority, expired perishables are shown at the top of the list first before longest unused, then cheapest.
- AI-generated summaries powered by Google gemini
- Falls back on hardcoded summary if unavailable.
- Forms have full input validation.

The SQL database contains:

- Items table which contains id, name, brand, price, purchase_date, last_used_date, status, category, manually_set values.
- Clothing table which contains id (foreign key), size, color values.
- Electronics table which contains id (foreign key), size, color values.
- Perishables table which contains id (foreign key), opened_date, shelf_life_days values.

Design Decisions

- Item is an abstract and uses inheritance, which means each subclass has its own data table containing its fields, which is linked to the items table by id. This prevents duplication of data, allowing OOP to be intact and the DeclutterService to work with any items that have a reference from Item.
- What certain classes do:
    - Inventory handles the storage and retrieval of items, and doesn't contribute to decision making
    - DeclutterService handles the flagging, suggestions, and prioritization of flagged items.
    - ItemController handles the passing and routing of data, and has no say on the logic. It doesn't know whether it is talking to a real AI advisor or a mock.
    - AIAdvisor is hidden so that the implementation of AI can freely switch without the need to alter other classes.
- Manually Setting Status of an item

    - Users can lock/manually set an item's status using a drop down on the main page (index.html). This causes the auto-suggest/update system to skip locked items entirely. By allowing this, users can override app recommendations while keeping the system usable. In addition, locked items that do not have "Keep" status will still appear on the flagged list as the user wants to take action on them.
- Auto-Updating Statuses
    - Item statuses are recalculated every time an item is saved, accessed the report page, or marked as used instead of being timed. This allows the database to be in sync with the data.
- Declutter Logic
    - Items are auto-flagged and auto-assigned statuses depending on the following:
        - Expired perishables are always in "Discard" status (regardless of other factors)
            - Perishables will expire if the difference between the current date and the opened date exceeds the shelf life days.
            - Can also occur if an unopened item hasn't been used in 1+ years.
        - Unused items of 30+ days and >= $50 is "Sell" status
        - Unused items of 30+ days and < $50 is "Donate" status
        - Recently used will always be in "Keep" status, unopened perishables will be checked if over a year has passed since their purchase date before doing so.
- Prioritized Removal logic on Suggested Removals table
    - Expired perishables are always first, most urgent
    - Longest unused next
    - Then cheapest if both items have the same unused time
- AI Integration
    - AIAdvisor is implemented by GeminiAIAdvisor and MockAIAdvisor. Swapping between them is located within ItemController.java. This allowed the AI to be testable and functional without an API key.
    - To use the actual AI an API Key from Google Gemini is required, and the key must be pasted in GeminiAIAdvisor.java in the line that mentions:
        - `private static final String API_KEY = YOUR_API_KEY_HERE";`

- AI's Role within this application
    - The AI generates a 2-3 sentence summary based on the user's inventory stats and items to remove. It generates based on these numbers and turns them into a readable message that backs up the report.
    - The AI receives the following:
        - Total items owned
        - Total money spent that is currently in the user's inventory
        - Number of flagged items
        - Money tied up in unused items
        - Names of priority items to remove

- There are times where the AI may fail to properly generate a response due to their rate limit. If this limit is exceeded, the app falls back to a hardcoded message rather than an error.
- Swapping to Mock AI and GeminiAI
  - In the ItemController.java, the line that says “private GeminiAIAdvisor aiAdvisor;” must be changed to “private MockAIAdvisor aiAdvisor;” to change to a mock AI. Vice versa for the other way around.
  - The ItemController automatically swaps to MockAI if the actual AI advisor is down however.
- Tools used
  - Spring Boot (web framework)
  - Spring Data JPA (to access the database)
  - Thymeleaf (for HTML templating)
  - MySQL via MAMP (storing values in database)
  - Google Gemini API (to generate AI summaries)
  - Gson (JSON parsing, Java library from Google, used in GeminiAIAdvisor.java)